

Bài thực hành làm quen với Javascript-02

Mục tiêu:

- Sử dụng JavaScript để chặn hành vi mặc định của Form.
- Viết hàm Validate dữ liệu (Email, Số điện thoại 10-11 số).
- Sử dụng fetch, .then(), .catch() để gửi dữ liệu lên Server.
- Xử lý trạng thái "Loading" (Đang gửi...) để nâng cao trải nghiệm người dùng.

Yêu cầu chuẩn bị:

- Sử dụng file index.html (chứa đoạn code cta-section ở bài thực hành landing page trước đó).
- Tạo file script.js và nhúng vào cuối thẻ <body> trong HTML.
- Đảm bảo form trong HTML có id là register-form và các input có id tương ứng (fullname, email, phone, course).

Bạn đã sẵn sàng bắt đầu hành trình học tập chưa?

Đừng bỏ lỡ cơ hội phát triển bản thân cùng TechAcademy. Hãy để lại thông tin để được tư vấn khóa học phù hợp nhất!

Đăng ký ngay

Phần 2: Viết hàm kiểm tra dữ liệu (Validation Logic) (20 phút)

Hướng dẫn: Trước khi xử lý sự kiện, hãy viết các "công cụ" (hàm) để kiểm tra tính đúng đắn của dữ liệu.

Sinh viên viết vào script.js:

```
// 1. Hàm kiểm tra Email hợp lệ (Sử dụng Regex)
```

```
function isValidEmail(email) {
    const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return regex.test(email);
}

// 2. Hàm kiểm tra Số điện thoại (Phải là số, độ dài 10-11 ký tự)
function isValidPhone(phone) {
    const regex = /^[0-9]{10,11}$/;
    return regex.test(phone);
}
```

Giải thích: Regex (Regular Expression) là mẫu quy tắc dùng để so khớp chuỗi. `[0-9]{10,11}` nghĩa là chỉ chấp nhận số từ 0-9 và phải lặp lại 10 đến 11 lần.

Phần 3: Xử lý sự kiện Submit (30 phút)

Nhiệm vụ:

1. Bắt sự kiện submit của form.
2. Dùng `e.preventDefault()` để chặn trang web tải lại.
3. Lấy giá trị từ các ô input.
4. Kiểm tra lỗi. Nếu có lỗi -> `alert()` báo lỗi và dừng chương trình.

Code mẫu (tham khảo):

```
const form = document.getElementById('register-form');

form.addEventListener('submit', function(e) {
    // Chặn reload trang
    e.preventDefault();
```

```
// Lấy giá trị
const name =
document.getElementById('fullname').value.trim();

const email = document.getElementById('email').value.trim();
const phone = document.getElementById('phone').value.trim();
const course = document.getElementById('course').value;

// --- VALIDATION ---
// Bước 1: Kiểm tra rỗng
if (!name || !email || !phone || !course) {
    alert("Vui lòng nhập đầy đủ thông tin!");
    return; // Dừng chạy tiếp
}

// Bước 2: Kiểm tra định dạng Email
if (!isValidEmail(email)) {
    alert("Email không đúng định dạng!");
    return;
}

// Bước 3: Kiểm tra định dạng Phone
if (!isValidPhone(phone)) {
    alert("Số điện thoại phải là số và có 10-11 chữ số!");
    return;
}

// Nếu code chạy đến đây nghĩa là dữ liệu đã OK
// Gọi hàm gửi dữ liệu (sẽ viết ở phần 4)
```

```
        submitData(name, email, phone, course);  
    });
```

Phần 4: Gọi API & Xử lý Bất đồng bộ (30 phút)

Nhiệm vụ: Viết hàm submitData để gửi dữ liệu đi. **Yêu cầu nâng cao:**

- Khi bắt đầu gửi: Đổi chữ nút bấm thành **"Đang xử lý.."** và vô hiệu hóa nút (để user không bấm liên tục).
- Khi gửi xong (thành công hoặc thất bại): Trả lại trạng thái nút ban đầu.

```
function submitData(name, email, phone, course) {  
    // 1. Tạo object dữ liệu  
    const dataToSend = {  
        name: name,  
        email: email,  
        phone: phone,  
        course: course  
    };  
  
    // 2. UX: Bật trạng thái Loading  
    const btnSubmit = form.querySelector('button');  
    const originalText = btnSubmit.innerText; // Lưu lại chữ gốc  
    btnSubmit.innerText = "Đang xử lý...";  
    btnSubmit.disabled = true; // Không cho bấm nữa  
  
    // 3. Gọi API (Fetch)  
    fetch('https://jsonplaceholder.typicode.com/posts', {  
        method: 'POST',  
        headers: {  
            'Content-Type': 'application/json'  
        }  
    })  
    .then(response => response.json())  
    .then(data => {  
        // Xử lý thành công hoặc thất bại  
        btnSubmit.innerText = originalText;  
        btnSubmit.disabled = false;  
    })  
    .catch(error => {  
        // Xử lý lỗi  
        btnSubmit.innerText = originalText;  
        btnSubmit.disabled = false;  
    });  
}
```

```
    },
    body: JSON.stringify(dataToSend)
  })
  .then(response => {
    // Kiểm tra nếu response có vấn đề (ví dụ lỗi 404, 500)
    if (!response.ok) {
      throw new Error("Lỗi mạng hoặc server bận");
    }
    return response.json();
  })
  .then(data => {
    // THÀNH CÔNG
    console.log("Server phản hồi:", data);
    alert(`Đăng ký thành công khóa học: ${course}\nChúng tôi sẽ liên hệ qua SĐT: ${phone}`);

    // Reset form
    form.reset();
  })
  .catch(error => {
    // THẤT BẠI
    console.error("Lỗi:", error);
    alert("Có lỗi xảy ra: " + error.message);
  })
  .finally(() => {
    // KẾT THÚC (Dù thành công hay thất bại đều chạy vào đây)
    // Trả lại trạng thái nút bấm ban đầu
```

```
        btnSubmit.innerText = originalText;

        btnSubmit.disabled = false;

    });
}
```

Phần 5: Yêu cầu thêm tự bổ sung

Hiển thị lỗi đẹp hơn: Thay vì dùng `alert()`, hãy tạo một thẻ `<p class="error-text" style="color: red"></p>` ngay dưới ô input bị lỗi và hiển thị nội dung lỗi vào đó. Khi người dùng nhập lại đúng thì ẩn dòng lỗi đi.